Gemini and CoPilot were both given the prompt below. Both GenAIs were accessed on a KU Linux machine using Firefox browser.

Prompt:

- “The attached C++ file contains a program that solves Sudoku puzzles from text files. When the program solves a puzzle, it writes the solution of the puzzle to a separate text file. Modify the program to instead write all of the puzzle solutions to a single text file named “solutions.txt”. For each puzzle solution, the following should be written to the file: A line stating the current puzzle “Puzzle: x” ,where x is the current puzzle number taken from the input file’s name. Then, a line stating “No solution found.” if the puzzle could not be solved. If the puzzle was solved, all possible solutions should be written to the file (in the same format as the input file), followed by a line stating “Solutions found: n”, where n is the number of solutions found. Clearly indicate modifications and additions with detailed and understandable comments.”

Gemini’s Program:

```
/*
Program name: Assignment6 (revision of Assignment5)
Program description:
   a C++ program that solves sudoku puzzles
   using an object oriented, recursive approach
   with depth-first search and backtracking
Inputs: txt file containing an unsolved 9x9 sudoku puzzle (puzzle1.txt - puzzle5.txt)
Outputs:
   the puzzle filename, the unsolved puzzle,
   the solved puzzle if 1+ solutions exist,
   "no solution" if no solutions exist
   solutions.txt (single output file containing all puzzle headers, solutions, and
counts)
Collaborators:
   "What is meant with "const" at end of function declaration?"

(https://stackoverflow.com/questions/3141087/what-is-meant-with-const-at-end-of-function-
declaration)
Other sources:
   Gemini (original Assignment 4 code)
   ChatGPT (assignmetnt 5 changes)
   Claude (assignment 5 improvements after changes)

Author: Ana Gonzalez Yuil
Creation Date: 10.14.26
```

```
Revision Date: 10.14.26
Revisions: personal comments (indicated by ***), check for enough filled cells (at least
17)

Revised by: Andy Garcia
Revision Date: 10/28
New revisions marked by REVISIONS:

Revised by: Ana Gonzalez Yuil
Revision Date: 11.10.26
Sources used: Gemini
Revisions: additional comments (for personal understanding)
*/

/**
* @file Assignment5.cpp
* @brief Object-based Sudoku Solver using Recursive Backtracking (DFS) in C++.
*
* This program reads partially solved Sudoku puzzles from text files where blank spaces
* are represented by underscores ('_'). It uses a class-based approach, storing the
* grid in nested vectors (lists), and solves each puzzle using depth-first search
* with backtracking. It outputs the filename, the unsolved puzzle, all solutions
* found, and the total solution count to a single "solutions.txt" file.
*/

#include <iostream>
#include <vector>
#include <string>
#include <fstream>

// Removed the <<cctype> include that ChatGPT added because it did not use it inside any
change. (Andy Garcia)
/**
* @class SudokuPuzzle
* @brief Represents a 9x9 Sudoku puzzle board and provides methods to load,
*        validate, and solve the puzzle using recursive backtracking.
*/
class SudokuPuzzle {
private:
```

```
    // 2D vector (list of lists) to store the 9x9 grid characters ('1'-'9' or '_')
    std::vector<std::vector<char>> grid;

public:
    /**
     * @brief Construct a new Sudoku Puzzle object.
     * Initializes a 9x9 grid filled with placeholder underscores ('_').
     */
    SudokuPuzzle() {
        grid.resize(9, std::vector<char>(9, '_')); // *** changes size of grid after
initialization
        // *** '9' sets the size of grid to 9.
        // *** 'std::vector<char>(9, '_')' initializes every element of grid to be a
vector of size 9 with every element set to '_'
    }

    /**
     * @brief Loads a Sudoku puzzle from a text file.
     *
     * @param filename The path/name of the file to open.
     * @return true If the file was successfully opened and read.
     * @return false If the file could not be opened.
     */
    bool loadFromFile(const std::string& filename) {// *** takes input variable by
reference (does not create a copy)
        int filledCells = 0; // *** holds amount of filled cells in unsolved sudoku grid

        //check if file exists/can be opened
        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        // Read 9 rows and 9 columns from the file
        for (int i = 0; i < 9; ++i) { // ** ++i works the same as i++
            for (int j = 0; j < 9; ++j) {
                // REVISION: Checks if the cell at the index can be read
                if (!(file >> grid[i][j])) {
                    file.close(); // REVISION: Closes the file if the character at the
```

```
index can't be read
                    return false; // REVISION: Rejects the file if it has less than 81
cell values
                }

                // REVISION: Checks and rejects anything that is not an _ or 1-9
                if (grid[i][j] != '_' &&
                    (grid[i][j] < '1' || grid[i][j] > '9')) {
                    file.close(); // REVISION: Closes the file before it returns
                    return false; // REVISION: Returns false to prevent bad characters
from being in the solver function
                }

                // *** implement check for at least 17 filled squares
                if (grid[i][j] != '_') {
                    filledCells++; // *** if the cell is not empty, add to filledCells
count.
                }
            }
        }

        // file.close();  CODE CHANGE: Removed this line to allow for the next checks to
happen (Andy Garcia)

        // REVISION: Reject extra values that exceed the 9x9 grid
        // if the outstream exceeds the expected dimensions, extraCharacter will not be
empty and therefore cause a return false
        char extraCharacter;
        if (file >> extraCharacter) {
            return false;
        }

        // REVISION: Keep the original filled cell check
        if (filledCells < 17) {
            return false; // Prevents empty input
        }

        // REVISION: Check to see if the values violate the Sudoku rules
        // modified the Revision to improve maintainability moving it to a new function
```

```
        if (!givensAreConsistent()) {
            return false; // two given numbers will break Sudoku rules
        }
        return true;
    }

    /**
     * @brief Prints the current state of the puzzle grid to standard output.
     */
    void print() const { // *** const keyword after function declaration makes it so that
class function cannot change the input (StackOverflow)
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                std::cout << grid[i][j] << " ";
            }
            std::cout << "\n";
        }
    }

    /**
     * @brief Validates if placing a specific character in a cell follows Sudoku rules.
     *
     * Checks if the value already exists in the given row, column, or 3x3 subgrid.
     *
     * @param row The row index (0-8) of the cell.
     * @param col The column index (0-8) of the cell.
     * @param val The character value ('1'-'9') to check.
     * @return true If the value is valid according to Sudoku rules.
     * @return false If the value violates row, column, or subgrid constraints.
     */
    bool isValid(int row, int col, char val) const {
        // 1. Check the row and column constraints
        for (int i = 0; i < 9; ++i) {
            // *** checks every num in row to see if value already exists
            if (grid[row][i] == val) return false; // Found in row
            // *** checks every num in col to see if value already exists
            if (grid[i][col] == val) return false; // Found in column
        }
```

```
        // 2. Check the 3x3 subgrid constraint
        // *** startRow and startCol calculation:
        // *** this snaps the start of the loop to the constraints of the corresponding
subgrid
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int i = 0; i < 3; ++i) {
            for (int j = 0; j < 3; ++j) {
                if (grid[startRow + i][startCol + j] == val) {
                    return false; // Found in the 3x3 subgrid
                }
            }
        }

        return true; // REVISION: Returning true instead of val to make it permanent and
improve readability
    }



    /**
     * @brief Checks that the given numbers do not already break a Sudoku rule.
     *        This is ChatGPT's conflict check, moved out of loadFromFile into its own
function.
     * @return true if no given number conflicts with another, false otherwise.
     */
    // Added givensAreConsistent() function. Code implemented by Andy Garcia suggested by
Claude.
    // checks that numbers do not repeat in compliance with sudoku rules
    bool givensAreConsistent() {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                if (grid[i][j] == '_') continue;  // only filled cells need checking
                char originalValue = grid[i][j];  // save the given before temporarily
clearing it
                grid[i][j] = '_';                 // exclude the given itself from the
checks
                bool valid = isValid(i, j, originalValue); // check it against its row,
column, and box
                grid[i][j] = originalValue;       // restore the given
```

```
                if (!valid) return false;          // two givens conflict
            }
        }
        return true;
    }

    /**
     * @brief Iterates through the grid to find the next empty cell.
     *
     * @param row Reference variable updated with the row index of the empty cell.
     * @param col Reference variable updated with the column index of the empty cell.
     * @return true If an empty cell ('_') is found.
     * @return false If the entire grid is filled (no empty cells remain).
     */
    bool findEmptyCell(int& row, int& col) const {
        for (row = 0; row < 9; ++row) {
            for (col = 0; col < 9; ++col) {
                if (grid[row][col] == '_') {
                    return true; // Empty cell located
                }
            }
        }
        return false; // Puzzle is completely filled
    }

    /**
     * @brief Recursive Depth-First Search function with backtracking to find all
solutions.
     *
     * Identifies an empty cell, iterates through candidate values ('1'-'9'),
     * places valid candidates, and recurses to the next empty cell. Backtracks
     * by resetting cells if a branch leads to a dead end.
     *
     * @param solutionCount Reference counter tracking the number of valid solutions
found.
     * @param solutions Reference container storing all valid solution grids.
     */
    void solveAll(int& solutionCount, std::vector<std::vector<std::vector<char>>>&
solutions) {
```

```
        int row, col;

        // Base case: If no empty cell remains, a complete valid solution is achieved
(k=81)
        if (!findEmptyCell(row, col)) {
            solutionCount++;
            solutions.push_back(grid);
            return;
        }

        // Try candidate values from '1' to '9' for the current empty cell
        for (char val = '1'; val <= '9'; ++val) {
            if (isValid(row, col, val)) {
                // Place candidate value into the cell
                grid[row][col] = val;

                // Move onto the next empty cell with a recursive function call
                solveAll(solutionCount, solutions);

                // Backtrack: Reset cell to empty if the recursive branch fails
                grid[row][col] = '_';
            }
        }
    }

    /**
     * @brief Helper function to process, solve, and write the results for a specific
puzzle file
     *        to the shared single output file stream ("solutions.txt").
     *
     * @param filename The file name containing the puzzle.
     * @param outFile Reference to the single shared output file stream.
     */
    static void processPuzzle(const std::string& filename, std::ofstream& outFile) {
        std::cout << "========================================\n";
        std::cout << "Puzzle Filename: " << filename << "\n";
        std::cout << "========================================\n";

        // MODIFICATION: Extract the current puzzle number 'x' from the input filename
```

```
        // (e.g., extracting '1' from "puzzle1.txt") and write "Puzzle: x" to
"solutions.txt".
        size_t startPos = 6; // Index where the number starts after prefix "puzzle"
        size_t endPos = filename.find('.', startPos);
        std::string puzzleNum = filename.substr(startPos, endPos - startPos);
        outFile << "Puzzle: " << puzzleNum << "\n";

        SudokuPuzzle puzzle;

        // Attempt to load the puzzle from the file
        if (!puzzle.loadFromFile(filename)) {
            std::cout << "Error: Could not open file " << filename
                    << " or puzzle input is invalid.\n\n";

            // MODIFICATION: Write "No solution found." to the single output file when
file loading fails or input is invalid.
            outFile << "No solution found.\n";
            return;
        }

        // Print the initial unsolved puzzle to console
        std::cout << "Unsolved Puzzle:\n";
        puzzle.print();
        std::cout << "\n";

        // Containers to store solutions and solution counts
        int solutionCount = 0;
        std::vector<std::vector<std::vector<char>>> solutions; // *** vector that holds
each solved sudoku grid

        // Execute backtracking solver
        puzzle.solveAll(solutionCount, solutions);

        // Print number of solutions to console
        std::cout << "Number of Solutions: " << solutionCount << "\n\n";

        // MODIFICATION: Write puzzle solution(s) or "No solution found." to the shared
"solutions.txt" file.
        if (solutionCount == 0) {
```

```
            std::cout << "no solution found\n\n";

            // Write "No solution found." if the puzzle could not be solved
            outFile << "No solution found.\n";
        } else {
            // Write all possible solutions in the same format as the input file
            for (size_t s = 0; s < solutions.size(); ++s) {
                std::cout << "Solution " << (s + 1) << ":\n";

                // Write each cell of the solution grid to the output file
                for (int i = 0; i < 9; ++i) {
                    for (int j = 0; j < 9; ++j) {
                        std::cout << solutions[s][i][j] << " ";
                        outFile << solutions[s][i][j] << " ";
                    }
                    std::cout << "\n";
                    outFile << "\n"; // Keep one row per line on the output file
                }

                std::cout << "\n";
                outFile << "\n"; // Separate consecutive solutions with a blank line
            }

            // MODIFICATION: Write "Solutions found: n" line after all solutions are
written
            outFile << "Solutions found: " << solutionCount << "\n";
        }
    }
};

/**
* @brief Main execution function.
*
* Opens the single "solutions.txt" output file, iterates through the five required
* puzzle files, processes each one sequentially, and closes the file stream.
*/
int main() {
    // Array containing the names of the five target text files
    std::string puzzleFiles[] = {
```

```
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // MODIFICATION: Open a single output file stream named "solutions.txt" for all puzzle
solutions.
    std::ofstream outFile("solutions.txt");
    if (!outFile.is_open()) {
        std::cerr << "Error: Could not open solutions.txt for writing.\n";
        return 1;
    }

    // Process each puzzle file sequentially, passing the shared output stream
    for (const std::string& filename : puzzleFiles) {
        SudokuPuzzle::processPuzzle(filename, outFile);
    }

    // MODIFICATION: Close the shared output file stream after all puzzles have been
processed.
    outFile.close();

    return 0;
}
```

Copilot's Program:

```
    /*
    Program name: Assignment6 (revision of Assignment5)
    Program description:
        a C++ program that solves sudoku puzzles
        using an object oriented, recursive approach
        with depth-first search and backtracking
    Inputs: txt file containing an unsolved 9x9 sudoku puzzle (puzzle1.txt - puzzle5.txt)
    Outputs:
        the puzzle filename, the unsolved puzzle,
        the solved puzzle if 1+ solutions exist,
```

```
       "no solution" if no solutions exist
       solution1.txt - solution5.txt (all solutions then "Solutions found: n", or the
line "No solution found.")
   Collaborators:
       "What is meant with "const" at end of function declaration?"

(https://stackoverflow.com/questions/3141087/what-is-meant-with-const-at-end-of-function-
declaration)
   Other sources:
       Gemini (original Assignment 4 code)
       ChatGPT (assignmetnt 5 changes)
       Claude (assignment 5 improvements after changes)

   Author: Ana Gonzalez Yuil
   Creation Date: 10.14.26
   Revision Date: 10.14.26
   Revisions: personal comments (indicated by ***), check for enough filled cells (at
least 17)

   Revised by: Andy Garcia
   Revision Date: 10/28
   New revisions marked by REVISIONS:

   Revised by: Ana Gonzalez Yuil
   Revision Date: 11.10.26
   Sources used:
   Revisions: additional comments (for personal understanding)
   */

   /**
    * @file Assignment5.cpp
    * @brief Object-based Sudoku Solver using Recursive Backtracking (DFS) in C++.
    *
    * This program reads partially solved Sudoku puzzles from text files where blank
spaces
    * are represented by underscores ('_'). It uses a class-based approach, storing the
    * grid in nested vectors (lists), and solves each puzzle using depth-first search
    * with backtracking. It outputs the filename, the unsolved puzzle, all solutions
    * found, and the total solution count.
```

```
 */

#include <iostream>
#include <vector>
#include <string>
#include <fstream>

// Removed the <<cctype> include that ChatGPT added because it did not use it inside any change. (Andy Garcia)
/**
 * @class SudokuPuzzle
 * @brief Represents a 9x9 Sudoku puzzle board and provides methods to load,
 *        validate, and solve the puzzle using recursive backtracking.
 */
class SudokuPuzzle {
private:
    // 2D vector (list of lists) to store the 9x9 grid characters ('1'-'9' or '_')
    std::vector<std::vector<char>> grid;

public:
    /**
     * @brief Construct a new Sudoku Puzzle object.
     * Initializes a 9x9 grid filled with placeholder underscores ('_').
     */
    SudokuPuzzle() {
        grid.resize(9, std::vector<char>(9, '_')); // *** changes size of grid after initialization
        // *** '9' sets the size of grid to 9.
        // *** 'std::vector<char>(9, '_')' initializes every element of grid to be a vector of size 9 with every element set to '_'
    }

    /**
     * @brief Loads a Sudoku puzzle from a text file.
     *
     * @param filename The path/name of the file to open.
     * @return true If the file was successfully opened and read.
     * @return false If the file could not be opened.
     */
```

```
      bool loadFromFile(const std::string& filename) {// *** takes input variable by
reference (does not create a copy)
          int filledCells = 0; // *** holds amount of filled cells in unsolved sudoku
grid

          //check if file exists/can be opened
          std::ifstream file(filename);
          if (!file.is_open()) {
              return false;
          }

          // Read 9 rows and 9 columns from the file
          for (int i = 0; i < 9; ++i) { // ** ++i works the same as i++
              for (int j = 0; j < 9; ++j) {
                  // REVISION: Checks if the cell at the index can be read
                  if (!(file >> grid[i][j])) {
                      file.close(); // REVISION: Closes the file if the character at the
index can't be read
                      return false; // REVISION: Rejects the file if it has less than 81
cell values
                  }

                  // REVISION: Checks and rejects anything that is not an _ or 1-9
                  if (grid[i][j] != '_' &&
                      (grid[i][j] < '1' || grid[i][j] > '9')) {
                      file.close(); // REVISION: Closes the file before it returns
                      return false; // REVISION: Returns false to prevent bad characters
from being in the solver function
                  }

                  // *** implement check for at least 17 filled squares
                  if (grid[i][j] != '_') {
                      filledCells++; // *** if the cell is not empty, add to filledCells
count.
                  }
              }
          }

          // file.close();  CODE CHANGE: Removed this line to allow for the next checks
```

```
to happen (Andy Garcia)

            // REVISION: Reject extra values that exceed the 9x9 grid
            // if the outstream execceds the expected dimensions, extraCharacter will not
be empty and therfore cause a return false
            char extraCharacter;
            if (file >> extraCharacter) {
                return false;
            }

            // REVISION: Keep the original filled cell check
            if (filledCells < 17) {
                return false; // Prevents empty input
            }

            // REVISION: Check to see if the values violate the Sudoku rules
            // modified the Revision to improve maintainability moving it to a new
function
            if (!givensAreConsistent()) {
                return false; // two given numbers will break Sudoku rules
            }
            return true;
        }

        /**
         * @brief Prints the current state of the puzzle grid to standard output.
         */
        void print() const { // *** const keyword after function declaration makes it so
that class function cannot change the input (StackOverflow)
            for (int i = 0; i < 9; ++i) {
                for (int j = 0; j < 9; ++j) {
                    std::cout << grid[i][j] << " ";
                }
                std::cout << "\n";
            }
        }

        /**
         * @brief Validates if placing a specific character in a cell follows Sudoku
```

```
rules.
         *
         * Checks if the value already exists in the given row, column, or 3x3 subgrid.
         *
         * @param row The row index (0-8) of the cell.
         * @param col The column index (0-8) of the cell.
         * @param val The character value ('1'-'9') to check.
         * @return true If the value is valid according to Sudoku rules.
         * @return false If the value violates row, column, or subgrid constraints.
         */
        bool isValid(int row, int col, char val) const {
            // 1. Check the row and column constraints
            for (int i = 0; i < 9; ++i) {
                // *** checks every num in row to see if value already exists
                if (grid[row][i] == val) return false; // Found in row
                // *** checks every num in col to see if value already exists
                if (grid[i][col] == val) return false; // Found in column
            }

            // 2. Check the 3x3 subgrid constraint
            // *** startRow and startCol calculation:
            // *** this snaps the start of the loop to the constraints of the
corresponding subgrid
            int startRow = row - (row % 3);
            int startCol = col - (col % 3);
            for (int i = 0; i < 3; ++i) {
                for (int j = 0; j < 3; ++j) {
                    if (grid[startRow + i][startCol + j] == val) {
                        return false; // Found in the 3x3 subgrid
                    }
                }
            }

            return true; // REVISION: Returning true instead of val to make it permanent
and improve readability
        }


        /**
```

```
        * @brief Checks that the given numbers do not already break a Sudoku rule.
        *        This is ChatGPT's conflict check, moved out of loadFromFile into its own
function.
        * @return true if no given number conflicts with another, false otherwise.
        */
       // Added givensAreConsistent() function. Code implemented by Andy Garcia suggested
by Claude.
       // checks that numbers do not repeat in compliance with sudoku rules
       bool givensAreConsistent() {
           for (int i = 0; i < 9; ++i) {
               for (int j = 0; j < 9; ++j) {
                   if (grid[i][j] == '_') continue;  // only filled cells need checking
                   char originalValue = grid[i][j];  // save the given before temporarily
clearing it
                   grid[i][j] = '_';                 // exclude the given itself from the
checks
                   bool valid = isValid(i, j, originalValue); // check it against its
row, column, and box
                   grid[i][j] = originalValue;       // restore the given
                   if (!valid) return false;         // two givens conflict
               }
           }
           return true;
       }

       /**
        * @brief Iterates through the grid to find the next empty cell.
        *
        * @param row Reference variable updated with the row index of the empty cell.
        * @param col Reference variable updated with the column index of the empty cell.
        * @return true If an empty cell ('_') is found.
        * @return false If the entire grid is filled (no empty cells remain).
        */
       bool findEmptyCell(int& row, int& col) const {
           for (row = 0; row < 9; ++row) {
               for (col = 0; col < 9; ++col) {
                   if (grid[row][col] == '_') {
                       return true; // Empty cell located
                   }
```

```
                }
            }
            return false; // Puzzle is completely filled
        }

        /**
         * @brief Recursive Depth-First Search function with backtracking to find all solutions.
         *
         * Identifies an empty cell, iterates through candidate values ('1'-'9'),
         * places valid candidates, and recurses to the next empty cell. Backtracks
         * by resetting cells if a branch leads to a dead end.
         *
         * @param solutionCount Reference counter tracking the number of valid solutions found.
         * @param solutions Reference container storing all valid solution grids.
         */
        void solveAll(int& solutionCount, std::vector<std::vector<std::vector<char>>>& solutions) {
            int row, col;

            // Base case: If no empty cell remains, a complete valid solution is achieved (k=81)
            if (!findEmptyCell(row, col)) {
                solutionCount++;
                solutions.push_back(grid);
                return;
            }

            // Try candidate values from '1' to '9' for the current empty cell
            for (char val = '1'; val <= '9'; ++val) {
                if (isValid(row, col, val)) {
                    // Place candidate value into the cell
                    grid[row][col] = val;

                    // Move onto the next empty cell with a recursive function call
                    solveAll(solutionCount, solutions);

                    // Backtrack: Reset cell to empty if the recursive branch fails
```

```
                grid[row][col] = '_';
            }
        }
    }

    /**
     * @brief Helper function to process, solve, and print the results
     *        for a specific puzzle file.
     *
     * @param filename The file name containing the puzzle.
     * @param outputFile Reference to the shared solutions.txt file.
     *
     * REVISION:
     * Added outputFile parameter so all puzzle results can be
     * written into a single output file instead of creating
     * separate solution#.txt files.
     */
    static void processPuzzle(const std::string& filename,
                              std::ofstream& outputFile)
    {
        std::cout << "========================================\n";
        std::cout << "Puzzle Filename: " << filename << "\n";
        std::cout << "========================================\n";

        // REVISION:
        // Extract the puzzle number from filenames such as:
        // puzzle1.txt -> 1
        // puzzle5.txt -> 5
        char puzzleNumber = filename[6];

        // REVISION:
        // Begin this puzzle's section in solutions.txt.
        outputFile << "Puzzle: " << puzzleNumber << "\n";

        SudokuPuzzle puzzle;

        // Attempt to load the puzzle from the file
        if (!puzzle.loadFromFile(filename))
        {
```

```
            std::cout << "Error: Could not open file "
                    << filename
                    << " or puzzle input is invalid.\n\n";

            // REVISION:
            // Required output when the puzzle cannot be solved
            // or the input file is invalid.
            outputFile << "No solution found.\n\n";
            return;
        }

        // Print the initial unsolved puzzle
        std::cout << "Unsolved Puzzle:\n";
        puzzle.print();
        std::cout << "\n";

        // Containers to store solutions and solution counts
        int solutionCount = 0;

        std::vector<std::vector<std::vector<char>>> solutions;

        // Execute backtracking solver
        puzzle.solveAll(solutionCount, solutions);

        // Print number of solutions
        std::cout << "Number of Solutions: "
                << solutionCount << "\n\n";

        if (solutionCount == 0)
        {
            std::cout << "no solution found\n\n";

            // REVISION:
            // Required output format for unsolved puzzles.
            outputFile << "No solution found.\n\n";
        }
        else
        {
            // Print and save every solution found.
```

```
            for (size_t s = 0; s < solutions.size(); ++s)
            {
                std::cout << "Solution "
                          << (s + 1)
                          << ":\n";

                for (int i = 0; i < 9; ++i)
                {
                    for (int j = 0; j < 9; ++j)
                    {
                        std::cout
                            << solutions[s][i][j]
                            << " ";

                        // REVISION:
                        // Write solution to solutions.txt
                        // using the same format as the input file.
                        outputFile
                            << solutions[s][i][j]
                            << " ";
                    }

                    std::cout << "\n";
                    outputFile << "\n";
                }

                std::cout << "\n";

                // REVISION:
                // Blank line separates multiple solutions.
                outputFile << "\n";
            }

            // REVISION:
            // Required final line showing how many
            // solutions were discovered.
            outputFile << "Solutions found: "
                       << solutionCount
                       << "\n\n";
```

```
            }
        }

    };

/**
* @brief Main execution function.
*
* Iterates through the required puzzle files and
* writes all results to a single file named solutions.txt.
*/
int main()
{
    // Array containing the names of the target files
    std::string puzzleFiles[] =
    {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // REVISION:
    // Open one shared output file for all puzzle solutions.
    std::ofstream outputFile("solutions.txt");

    // Ensure the file opened successfully.
    if (!outputFile.is_open())
    {
        std::cout
            << "Error: Could not create solutions.txt\n";

        return 1;
    }

    // REVISION:
    // Send the same output stream to every puzzle so
    // all results are written into solutions.txt.
```

```
    for (const std::string& filename : puzzleFiles)
    {
        SudokuPuzzle::processPuzzle(filename,
                                    outputFile);
    }

    // REVISION:
    // Close the shared output file after every
    // puzzle has been processed.
    outputFile.close();

    return 0;
}
```

Analysis:

1. Gemini
    a. Correctness:
        i. Gemini modifies the processPuzzle() and main() functions to print to one collective solutions file. However, the program also prints the same results to the console, which may be unwanted behavior.
        ii. When given expected input: The program still solves the sudoku puzzles as expected using a DFS algorithm with backtracking. The output file is named properly, and the solutions are formatted properly according to the prompt and the instructions, however, there is less separation between lines and solutions, which can affect readability.
        iii. When given unexpected input (file does not exist, dimensions are not 9x9, sudoku grid with no empty spaces[solved]): The program responds in the same way for the first two cases; the user is informed through an error written to the console and the file, and then program moves on to the next file. The console output reads “Error: Could not open file puzzle.txt or puzzle input is invalid.”, while the file output reads “No solution found.” In the case where the puzzle is already solved, the program uses the given puzzles as the solution.
    b. Execution Time [O(n)]:
        i. ProcessPuzzle()
            1. The execution time depends on the recursion of the sudoku solver, which depends on the number of empty cells needed to solve for (n complexity).
    c. Space Complexity [O(n)]:
        i. ProcessPuzzle()

1. The execution time depends on the number of solutions found, which affects the size of the solutions vector.

d. Maintainability:
   i. There is an adequate number of comments clearly indicating modifications and reasons, which makes the overall code well documented.
   ii. Since the function only returns an outstream and does not write directly to the file, it would be easy to implement more additions to the file via other functions in the future if necessary.
   iii. The implementation for obtaining the number from the puzzle input file accounts for more than one digit, which accounts for a larger amount of files in the future.

2. CoPilot
   a. Correctness:
      i. CoPilot also modifies processPuzzle() and main() to print to one collective file but prints to console along with writing to file (which, again, may be unwanted behavior).
      ii. When given expected input: The program still solves the sudoku puzzles as expected using a DFS algorithm with backtracking. The output file is named properly, and the solutions are formatted properly according to the prompt and the instructions. There is more separation between lines and solution in the solutions files, which is more readable.
      iii. When given unexpected input (file does not exist, dimensions are not 9x9, sudoku grid with no empty spaces[solved]): The program responds in the same way for the first two cases; the user is informed through an error written to the console and the file, and then program moves on to the next file. The console output reads “Error: Could not open file puzzle.txt or puzzle input is invalid.”, while the file output reads “No solution found.” In the case where the puzzle is already solved, the program uses the given puzzles as the solution.
   b. Execution Time [O(n)]:
      i. The execution time depends on the recursion of the sudoku solver, which depends on the number of empty cells needed to solve for (n complexity).
   c. Space Complexity [O(n)]:
      i. The execution time depends on the number of solutions found, which affects the size of the solutions vector.
   d. Maintainability:
      i. Comments documenting changes made are detailed in their explanation and properly indicate where modifications occur.

Program Selected: Gemini

1. Justification
   a. Gemini’s implementation of writing all output to one file is simpler and has better structure.

    b. The method for extracting the file number from the input file accounts for more cases compared to Copilot. Gemini does not assume that the number will always be in the 6th index position, or that it will always be a single digit.
2. Changes Made
    a. Omitted printing to terminal, as all output should be in one file.
    b. Added dividers/newlines into output for easier readability.
    c. Additional comments for explanation and understanding
    d. Added differentiation between reasons for no puzzle solution; indicates if file was unopenable or puzzle input was invalid instead of generic “no solution” line written to file.